Requisitos

"Não tem sentido ser preciso quando você ainda não sabe sobre o que se está falando".

Von Neumann

Engenharia de Software

O que é um requisito?

"Something required; something wanted or needed".

Webster's Ninth New Colegiate Dictionary

(1) "A condition or capability that must be met or processed by a system (...) to satisfy a contract, standard, specification, or other formally imposed document".

(2) "A condition or capability needed by a user to solve or achieve an objective".

IEEE 729

O que é um requisito?

- Um requisito pode variar desde uma sentença, em alto nível, indicando um serviço ou uma restrição de um sistema até uma especificação matemática detalhada.
- Os requisitos podem servir, entre outros aspectos, para:
  - Pode ser a base para a formulação de uma proposta a um contrato (lado do proponente).
  - Pode ser a base do contrato propriamente dito (lado do contratante)

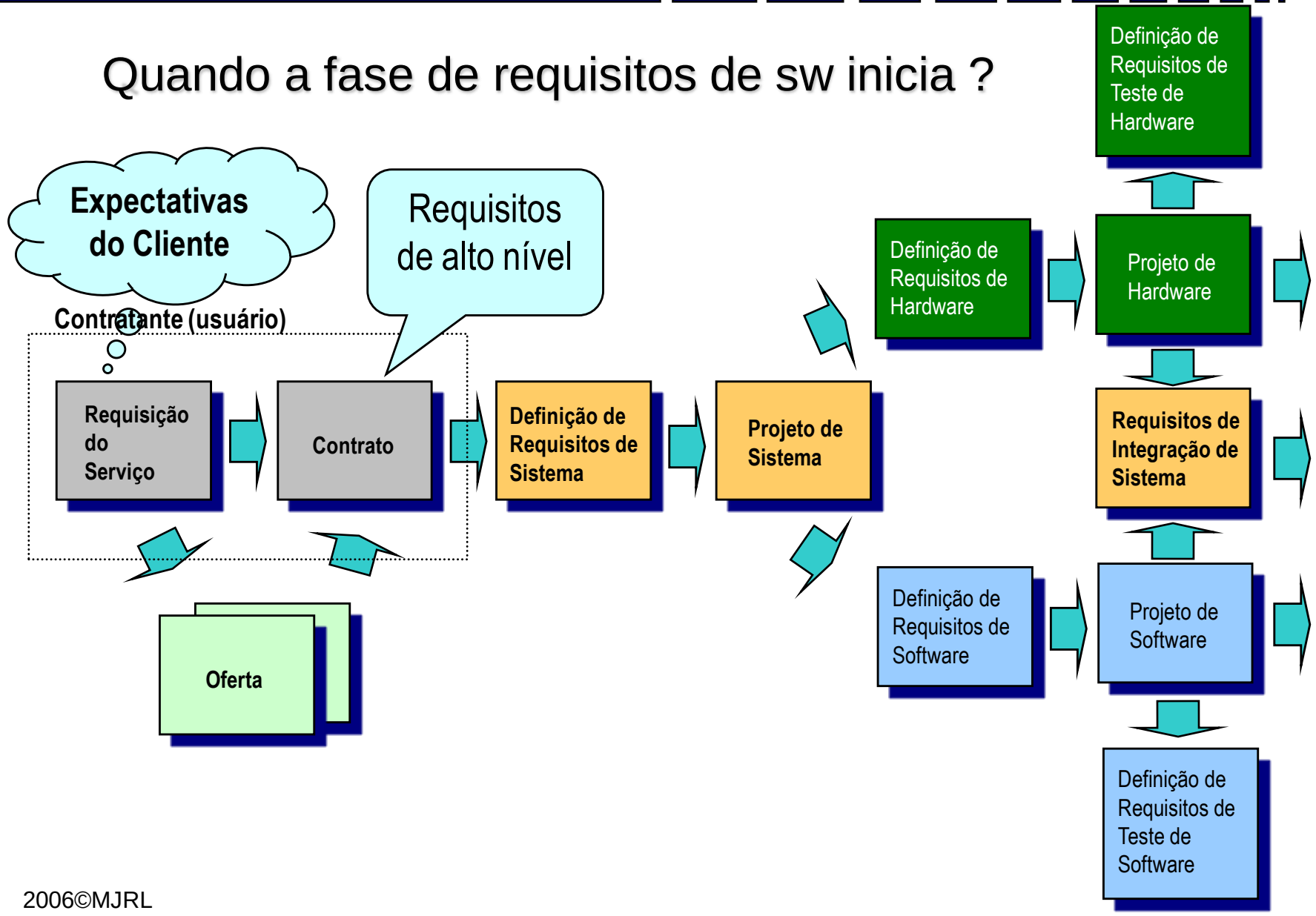
Engenharia de Requisitos

- O processo de estabelecer os serviços que o cliente necessita de um sistema e as restrições sob as quais ele opera e se desenvolve "

- "Os próprios requisitos são as descrições dos serviços do sistema e as restrições que são gerados durante o processo de engenharia de requisitos"
  Sommerville

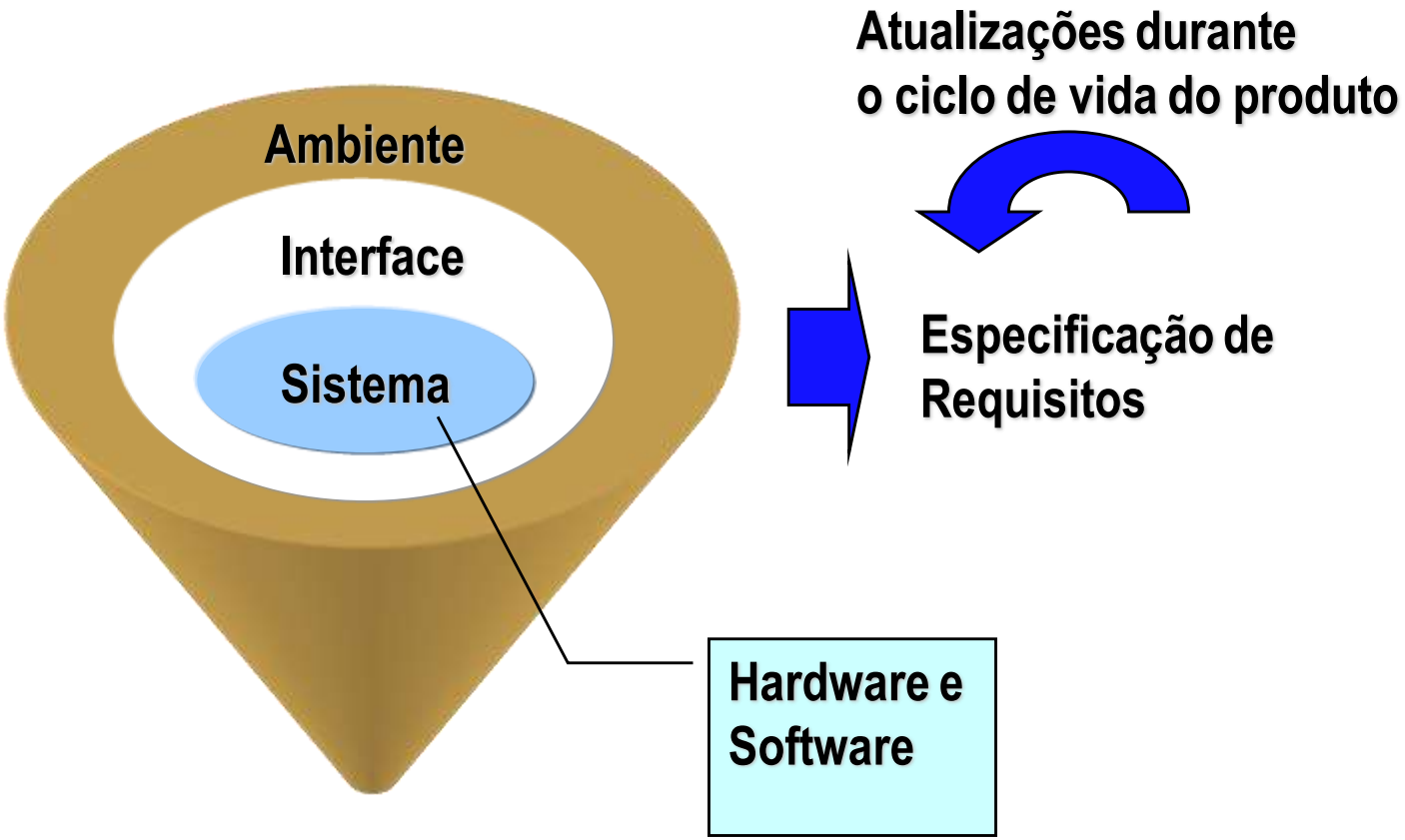
Engenharia de Software

Quando a fase de requisitos de sw inicia ?



Engenharia de Software

Escopo de uma definição de requisitos



Requisitos – quanto a origem

- **Requisitos do Usuário**
  - Texto em liguagem natural, diagramas do sistema e suas restrições operacionais. Escritos pelo usuário.
- **Requisitos do Sistema**
  - Documento estruturado apresentando uma descrição detalhada dos serviços que o sistema deve oferecer. Escrito como um contrato entre o cliente e o contratado.
- **Requisitos de Software**
  - Descrição detalhada do software que serve como base para o projeto e implementação. Escrito pelos desenvolvedores.

Requisitos – quanto ao tipo

- **Requisitos Funcionais**
  - Especificação dos serviçoes que o sistema deve prover, como o sistema deve reagir a certos inputs, como deveria ser o comportamento do sistema em certas situações.

- **Requisitos Não-Funcionais**
  - Restrições aos serviços ou funções oferecidas pelo sistema tais como restrições de tempo, restrições quanto ao processo de desenvolvimento, padrões, etc.

Requisitos Funcionais

- Descrevem as funcionalidade e serviços que o sistema deve oferecer.
- Dependem do tipo do software, potenciais usuários e do tipo de sistema onde o software será utilizado.
- Requisitos funcionais do usuário podem ser de alto nível porém os requisitos funcionais do sistema devem ser especificados em detalhes.

Exemplos de Requisitos Funcionais

- O usuário deve ser capaz de realizar pesquisas em toda a base de dados do sistema.
- O sistema deve permitir o acesso somente de usuários cadastrados.
- O ajuste de temperatura comandado pelo sistema deve ser limitado em 35° C.

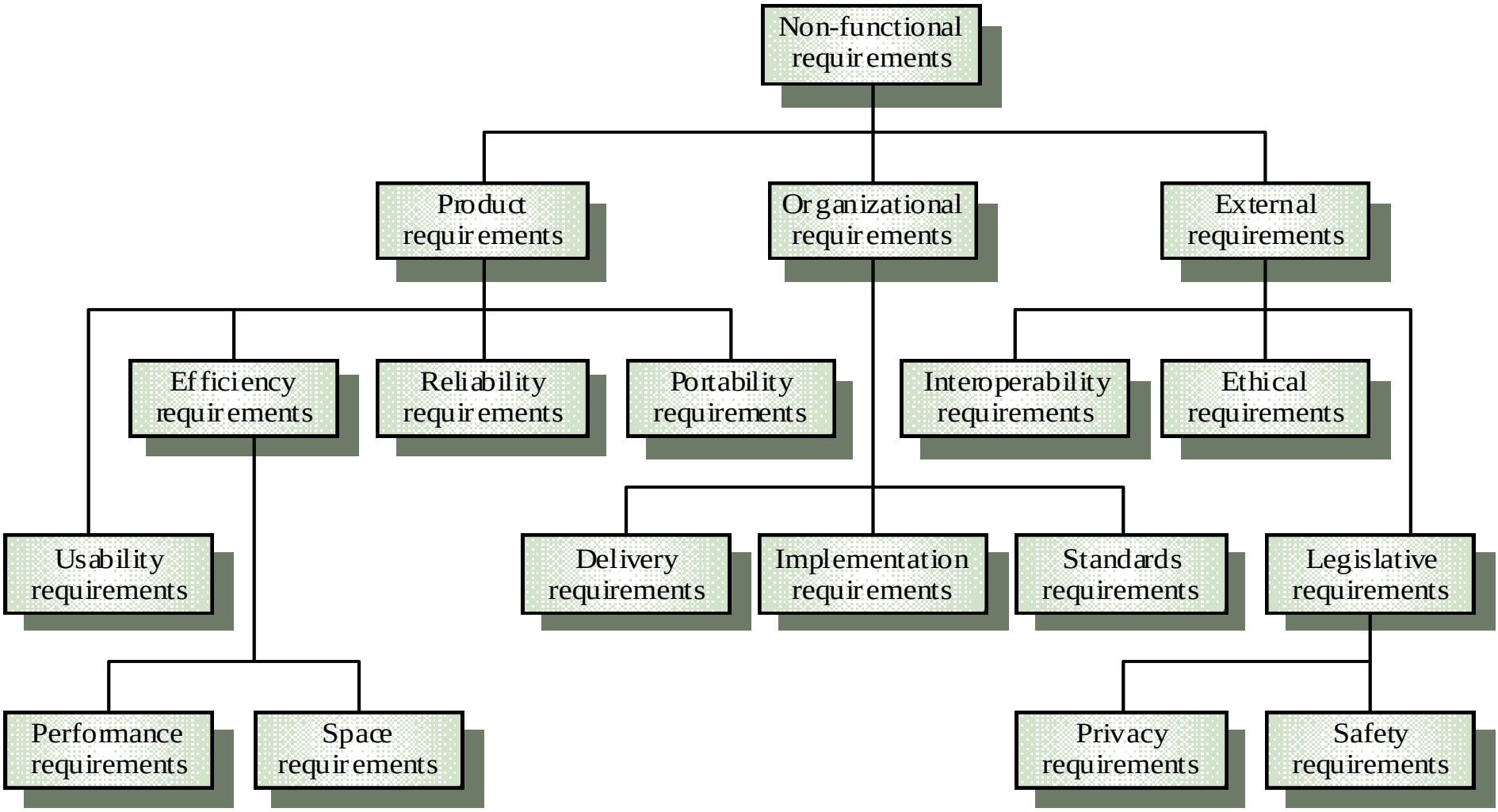
Requisitos Não-Funcionais

- Definem as propriedades do sistema e suas restrições, isto é, confiabilidade, tempo de resposta. Restrições podem ser capacidade de dispositivos de I/O, representações gráficas, etc.
- Requisitos de processo podem ser também especificados baseados em uma ferramenta CASE, liguagem de programação ou método de desenvolvimento.
- Requisitos Não-Funcionais podem ser mais críticos do que os Funcionais. Se estes não são atendidos o sistema pode perder sua utilidade.

Classificação para os Requisitos Não-Funcionais

- Requisitos de Produto
  - Requisitos que especificam que o sistema deve se comportar de uma determinada maneira, isto é, tempo de execução , confiabilidade, segurança, etc.
- Requisitos Organizacionais
  - Requisitos que são conseqüência de políticas organizacionais e procedimentos, isto é, uso de normas, regras de codificação, processos padronizados, etc.
- Requisitos Externos
  - Requisitos que surgem de fatores que são externos ao sistema e seu processo de desenvolvimento, isto é, requisitos de interoperabilidade, requisitos legais, etc.

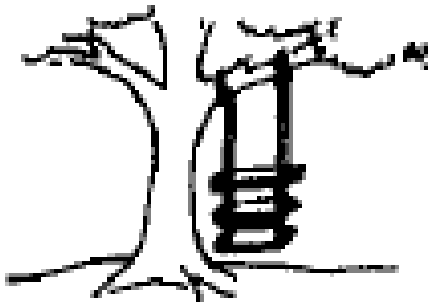
Tipos de Requisitos Não-Funcionais (Sommerville)



Exemplos de Requisitos Não-Funcionais

- Requisitos de Produto
  - O ciclo de execução da aplicação deve ser de 100ms.
- Requisitos Organizacionais
  - A elaboração da documentação do sistema/software deve ser de acordo com o procedimento interno XPTO-DOC-001.
- Requisitos Externos
  - O sistema deve atender a requisitos de segurança conforme o especificado no padrão RTCA-DO-178B.

What the Users Wanted...



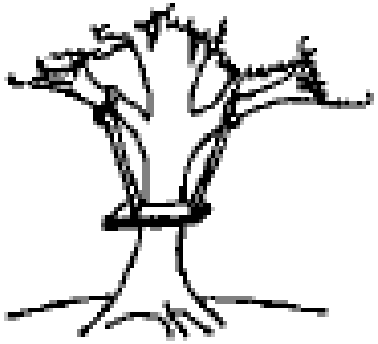
1. As proposed by Project Sponsor



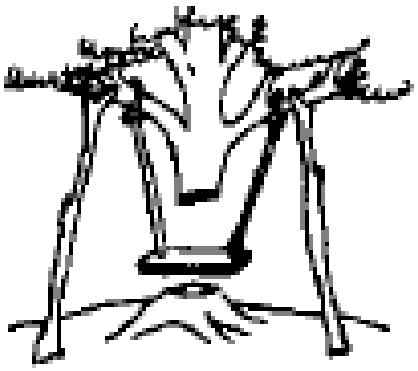
2. As specified in Project Requirements



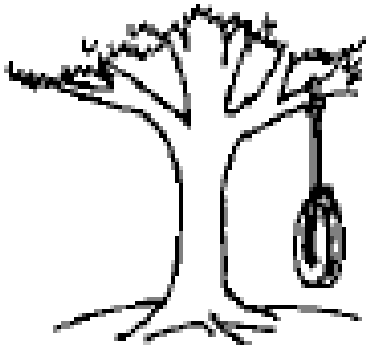
3. As designed by System Architect



4. As produced by Programmers



5. As installed



What the Users wanted

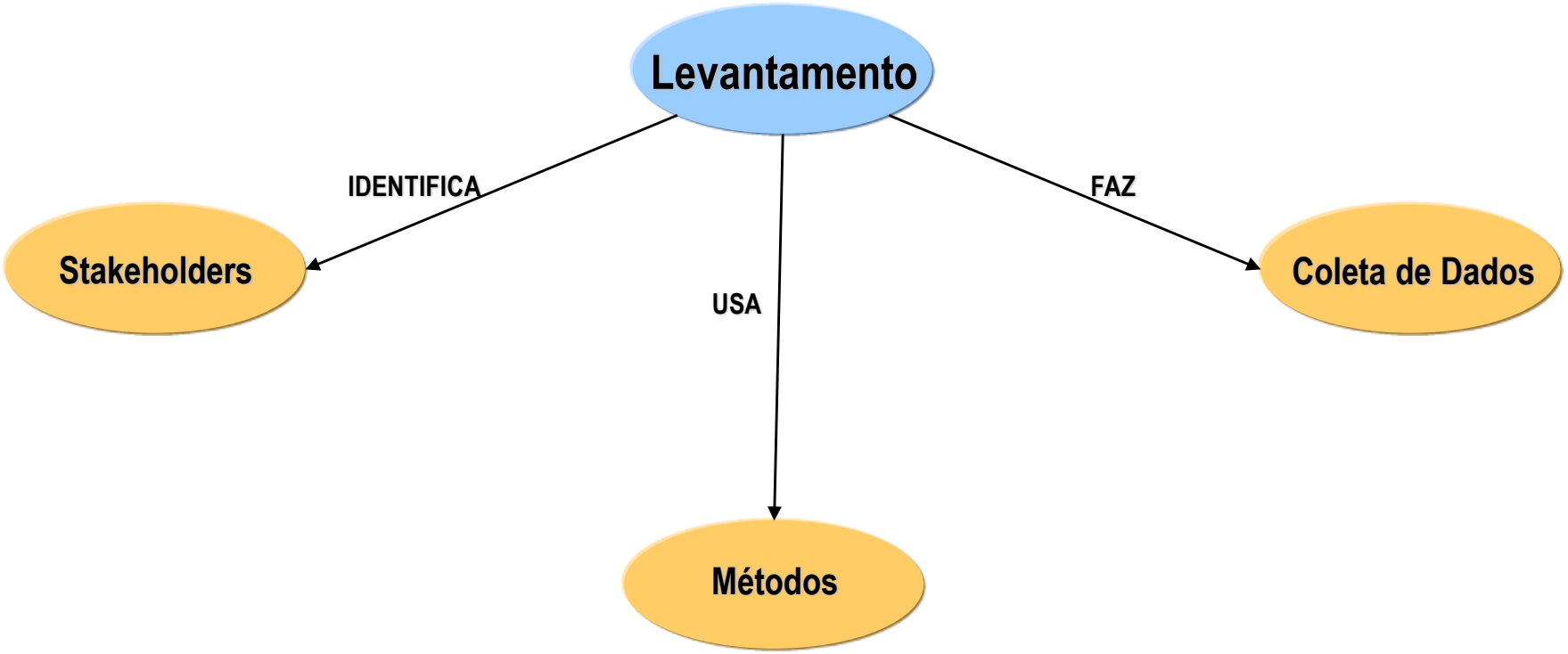
© LogOn Technology Transfer 1996

20

Fases

- A definição de requisitos pode ser encarada como constituída por três atividades:
  - **Levantamento (Elicitação)**
  - **Especificação**
  - **Modelagem**

Levantamento



Stakeholders

- Em um processo de levantamento de requisitos é fundamental identificar quem são os usuários do sistema que será desenvolvido (Stackholders).
- Esses stackeholders são as pessoas que irão fornecer as informações sobre suas expectativas sobre o sistema.
- A partir dessas informações serão extraídos os requisitos do sistema.

Stakeholders

- Regra prática:
  - Identifique os stakeholders
  - Identifique fontes de informação (documentos, normas, procedimentos, ...)
  - Procure entender, mesmo que de maneira preliminar, o problema do cliente
  - Prepare a conversa com os stakeholders (questões, local, agenda)
  - Anote o máximo que puder. Nada é irrelevante nesse momento
  - Debata com sua equipe os dados coletados e se for necessário reagende uma nova conversa com os stakeholders

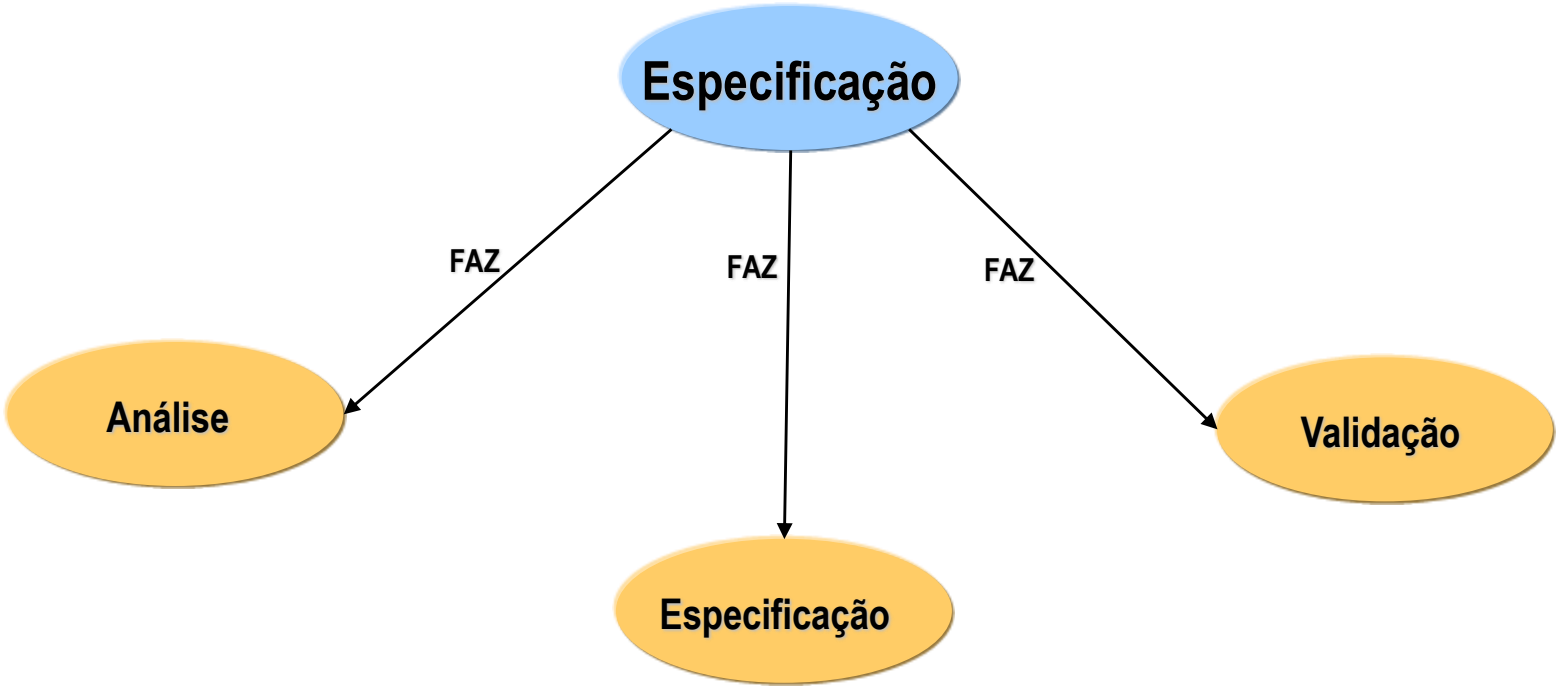
Coleta de Dados

- Os stakeholders são a fonte principal de informação para os requisitos, porém outros fontes devem também ser consideradas, tais como:
  - Documentação de sistemas existentes
  - Normas internas da empresa
  - Legislação, etc
- Essa informações devem ser coletadas e organizadas para análise e especificação dos requisitos.

Métodos

- Existem diversos métodos para o levantamento de requisitos. Dentre eles pode-se destacar:
  - Análise de documentos
  - Entrevistas
  - Reuniões
    - *Joint Application Development* (JAD)
  - O cliente escreve os requisitos (XP)
  - Prototipação

Especificação



Análise

- As informações coletadas e classificadas na etapa de elicitação precisam ser analisadas para se extrair os requisitos que o sistema deve atender.
- Em geral as informações que vêm do cliente estão em alto nível e é, muitas vezes, necessário detalhá-las.

Análise

- A análise das informações deve ser feita sob duas perspectivas:
  - Estática
    - » Cada informação deve ser compreendida individualmente.
      Ex.: Do que é constituído o cadastro de um funcionário e qual o significado para cada dado desse cadastro.
  - Dinâmica
    - » A informação deve ser compreendida dentro do contexto da sua operação.
      Ex.: Em momentos do processo de gestão de pessoal o cadastro de funcionário é utilizado.

Princípios de Análise – Análise domínio dos dados

- Define os objetos de dados
- Descreve os atributos dos dados
- Estabelece o relacionamento entre os dados

Princípios de Análise – Análise das funções

- Identificar funções que transformam os dados
- Identificar como os dados fluem através do sistema
- Representar os produtores e consumidores dos dados

Princípios de Análise – Análise comportamental

- Indicar os diferentes estados do sistema
- Especificar eventos que provocam mudanças de estado no sistema

Princípios de Análise – Particionamento

- Refinar cada modelo para representar níveis de abstração
  - Refinar os dados
  - Criar uma hierarquia funcional
  - Representar o comportamento do sistema em diferentes níveis de detalhes

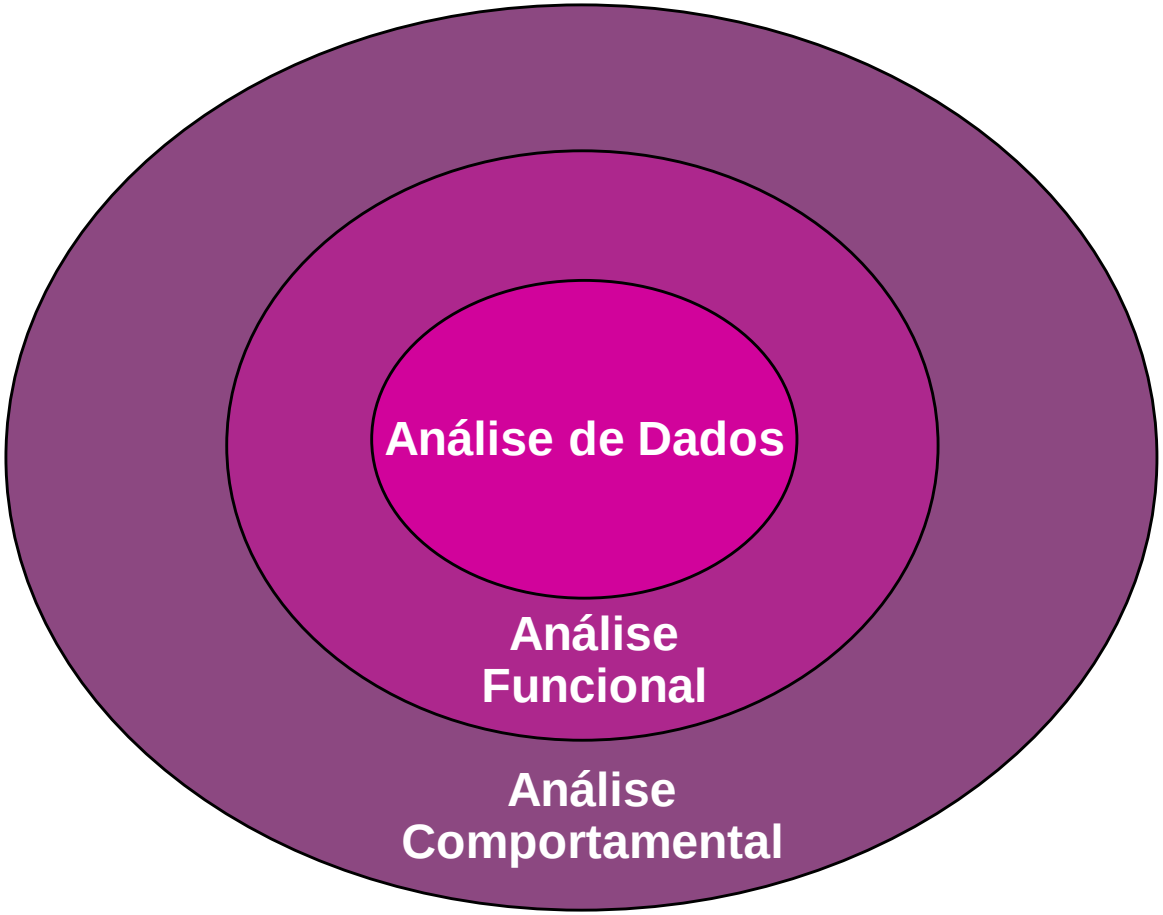
Princípios de Análise – Análise essencial

- Analise o sistema focando na essência do problema sem abordar detalhes de implementação

Princípios de Análise – Allan Davis

- Entenda o problema antes de começar a análise dos modelos
- Desenvolva protótipos que permitam ao usuário como as interações humanas-computador irão ocorrer
- Registre a origem e a razão para cada requisito
- Utilize multiplas visões dos requisitos
- Priorize os requisitos
- Trabalhe para eliminar a ambiguidade

Princípios de Análise – Modelo de análise



Especificação

- A especificação de requisitos consiste em identificar dentro do universo das informações analisadas o que o sistema a ser desenvolvido deve fazer.
- Por identificação quer se dizer a formalização do requisito em um formato previamente definido.
- As formas de especificação de requisitos variam bastante quanto ao seu grau de rigor. Os próximos slides apresentam formas alternativas para a especificação de requisitos.

Especificação – Linguagem Natural

- A linguagem natural é a forma mais simples de especificação de requisitos, porém apresenta os seguintes problemas:
  - Falta de Clareza
    - » Em geral é difícil se obter precisão sem que o documento se torne de difícil leitura.
  - Confusão de Requisitos
    - » Requisitos funcionais e não funcionais tendem a estar misturados.
  - Condensação de Requisitos
    - » Muitos requisitos diferentes podem estar expressos juntos.
  - Perda de Requisito
    - » Devido a uma combinação dos fatores acima, a probabilidade de não identificação de um requisito aumenta.

Especificação – Alternativas à Linguagem Natural

- Linguagem Estruturada
  - Uma forma limitada da linguagem natural pode ser usada para expressar requisitos.
  - Isso remove alguns dos problemas resultantes da ambiguidade e flexibilidade e impõe um grau de uniformidade à especificação.
  - Oferece suporte ao uso de formulários.

Especificação – Alternativas à Linguagem Natural

- Especificação Baseada em Formulários:
  - Definição da função ou entidade
  - Descrição das entradas e de onde elas vêm
  - Descrição das saídas e para onde elas vão
  - Indicação de outras entidades requeridas
  - Pre é pós condições (se apropriado)
  - Efeitos colaterais (se existirem)

Especificação – Alternativas à Linguagem Natural

Formulário - Exemplo

Função: Adicionar Nó

Descrição: Adicionar um nó a um projeto existente. O usuário deve selecionar o tipo do nó e sua posição. Quando adicionado ao projeto, o nó deve tornar-se o objeto correntemente selecionado. O usuário deve escolher a posição do nó movimentando o cursor até a área desejada.

Entradas: Tipo do nó, posição, identificador do objeto

Fonte: O tipo do nó e a sua posição devem ser fornecidos pelo usuário. O identificador do objeto deve ser armazenado em uma base de dados.

Saída: Identificador do objeto.

Pré-condições: O projeto é aberto e apresentado sua tela ao usuário.

Pós-condições: O projeto só é modificado no que se refere a adição do novo objeto.

Efeitos Colaterais: Nenhum.

Especificação – Alternativas à Linguagem Natural

- Especificação Baseada em Sentenças:
  - Acomoda a notação por formulários.
  - Define cada requisito, funcional ou não-funcional individualmente.
  - Facilita a identificação de que o requisito foi especificado e posteriormente atendido.

Especificação – Alternativas à Linguagem Natural

Sentenças - Exemplo

7.1 O sistema deve permitir ao usuário adicionar um objeto do tipo Nó

7.1.1 O sistema deve permitir ao usuário adicionar um nó a um projeto existente.

7.1.2 O usuário deve selecionar o tipo do nó e sua posição.

7.1.3 Quando adicionado ao projeto, o nó deve tornar-se o objeto correntemente selecionado.

7.1.4 O usuário deve escolher a posição do nó movimentando o cursor até a área desejada.

7.1.5 O objeto criado deve receber um identificador único

7.1.6 O identificador do objeto deve ser armazenado em uma base de dados

Restrições / Considerações

1. O projeto é aberto e apresentado sua tela ao usuário.
2. O projeto só é modificado no que se refere a adição do novo objeto.

Especificação – Alternativas à Linguagem Natural

- Programming Definition Language (PDL).
  - Uma PDL é como uma linguagem de programação contudo com maior flexibilidade para a construção de expressões.
  - É mais indicada quando
    - » Uma operação é indicada como uma seqüência de ações e a ordem é importante
    - » As interfaces entre hardware e software tenham que ser especificadas
  - As desvatagens são
    - » A PDL pode não ser suficiente para definir todos os conceitos do domínio do problema
    - » As especificações serão consideradas como o projeto e não como especificações

Especificação – Alternativas à Linguagem Natural

PDL - Exemplo

```
class ATM {
        // declarations here
        public static void main (String args[]) throws InvalidCard {
                try {
                        thisCard.read () ;  // may throw InvalidCard exception
                        pin = KeyPad.readPin () ; attempts = 1 ;
                        while ( !thisCard.pin.equals (pin) & attempts < 4 )
                                {       pin = KeyPad.readPin () ;  attempts = attempts + 1 ;
                                }
                                if (!thisCard.pin.equals (pin))
                                        throw new InvalidCard ("Bad PIN");
                        thisBalance = thisCard.getBalance () ;
                        do {  Screen.prompt (" Please select a service ") ;
                                service = Screen.touchKey () ;
                                switch (service) {
                                        case Services.withdrawalWithReceipt:
                                                receiptRequired = true ;
```

Especificação – Alternativas à Linguagem Natural

- Linguagens de Especificação Formal
  - As linguagens de especificação formal se utilizam de notação matemática para descrever de maneira precisa as propriedades que as informações de um sistema devem ter.
  - Existem diversas linguagens de especificação formal disponíveis, contudo a liguagem Z talvez seja a mais conhecida entre elas.

Especificação – Alternativas à Linguagem Natural

Z - Exemplo

AddThenFindBirthday

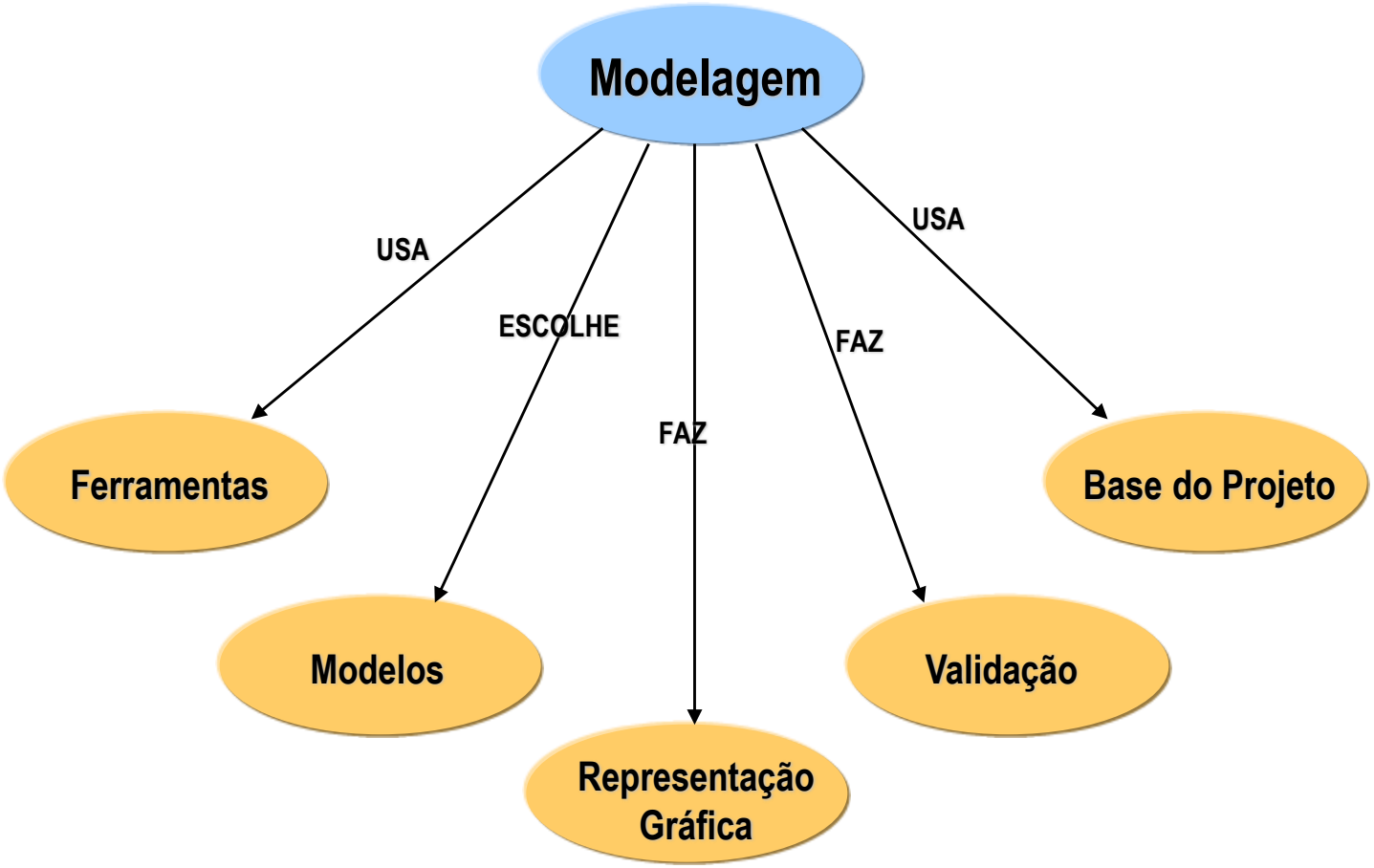
$$known : \mathbb{P}\, NAME$$
$$birthday : NAME \pfun DATE$$
$$known' : \mathbb{P}\, NAME$$
$$birthday' : NAME \pfun DATE$$
$$name? : NAME$$
$$date? : DATE$$
$$date! : DATE$$

$$\exists known'' : \mathbb{P}\, NAME;\ birthday'' : NAME \pfun DATE \bullet$$
$$known = \mathrm{dom}\, birthday \land$$
$$known'' = \mathrm{dom}\, birthday'' \land$$
$$name? \notin known \land$$
$$birthday'' = birthday \cup \{name? \mapsto date?\} \land$$
$$known'' = \mathrm{dom}\, birthday'' \land$$
$$known' = known'' \land$$
$$birthday' = birthday'' \land$$
$$name? \in known'' \land$$
$$date! = birthday''(name?)$$

Validação

- Antes de seguir para a próxima etapa do processo os requisitos precisam ser validados.
- Validar neste contexto significa garantir que os requisitos estão corretos e completos.
- O processo de validação pode variar. A validação pode ser feita, por exemplo, através de uma reunião de especialistas ou através da aplicação de um checklist.

Modelagem


Modelagem
USA
ESCOLHE
FAZ
FAZ
USA
Ferramentas
Modelos
Representação Gráfica
Validação
Base do Projeto

Modelagem

- O desenvolvimento de software é uma atividade complexa.
- Manter sob controle essa complexidade é elemento fundamental para o êxito de um processo de desenvolvimento.
- Um mecanismo que pode ser usado para tanto é a modelagem.

Modelagem

- A modelagem visa a representação de um sistema em um nível de abstração alto.
- Essa abstração em geral se dá através da utilização de elementos gráficos cuja semântica é regida por um método.
- Em síntese a modelagem tem por objetivo representar um problema de maneira a simplificar o entendimento do mesmo.

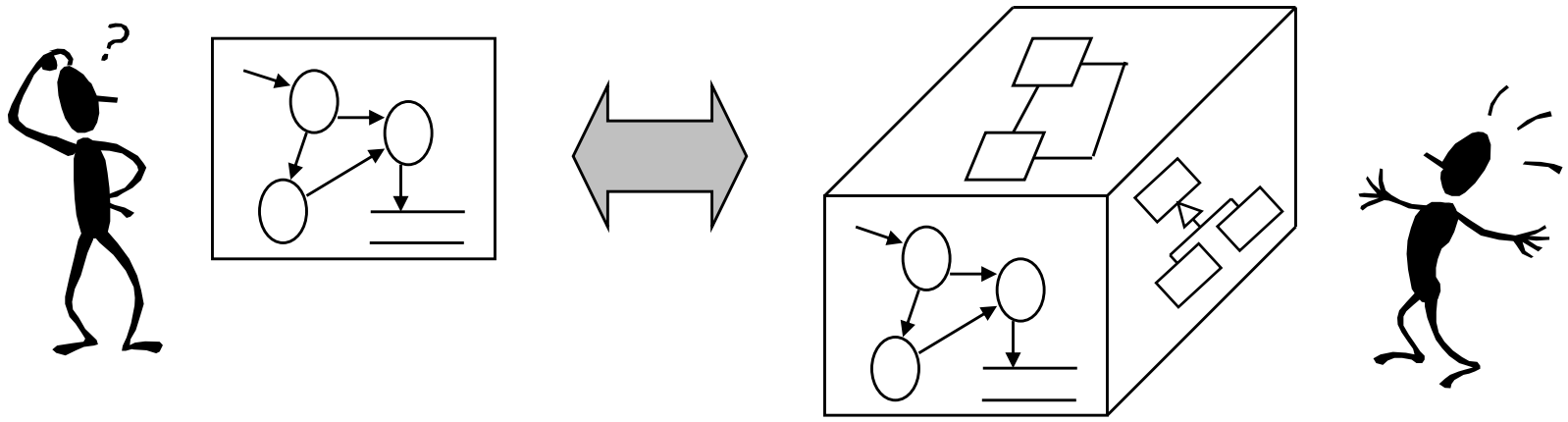
Modelagem

- Existem diversos métodos de modelagem, cada um deles com características diferentes e suportando diferentes tipos de análises.

- Em geral os métodos são compostos por diferentes tipos de diagramas e formas de representação de informações que, quando olhados em conjunto proporcionam uma visão do sistema.

Modelagem

- Não é possível obter uma visão completa do sistema analisando-se somente um tipo de representação.



Modelagem

- Existem diversos tipos de representações, como por exemplo:
  - Diagramas de Fluxo de Dados
  - Diagramas de Estados
  - Diagramas de Casos de Usos
  - Diagramas de Classe
  - Fluxogramas
  - ...